Data Creators › Day 2 — run sheet

Day 2 run sheet — Mappings, service levels and implementation foundations

Instructor-only companion to the anchor notebook, in the notebook's order: the same numbered sections and labelled parts, each with its time, purpose, what is on screen, what to say, what to ask at the whiteboard (with the answers rooms give and what to add), the demo steps with their coaching, fallbacks, the transition and what to do if ahead or behind. Keep the notebook on the projector and this beside it. Times follow the instructor plan; adapt them to the room and keep the sequence.

Sections

#	Section	Time	What happens
1	Mappings and service commitments	09:00–10:15	Demo: STTM and service levels · Activity 6: extend the contract · Break · Shareback: review and present a contract

© 2026 Decoded Limited · decoded.com

#	Section	Time	What happens
2	Platform architecture	10:15–11:00	Demo: warehouse, lake, lakehouse · Demo: Databricks under the hood, Delta, Spark · Demo: how the tools fit together · Shareback · Break
3	Docker	11:00–13:00	Demo: the five steps and the dev container's Dockerfile · Challenge 7: Docker practical · Lunch
4	Contract to code	13:00–15:00	Demo: where output lands, live generation · Activity 8: generate and investigate · Break · Debrief · Activity 8b: review another pair's code · Close

Timeline

§	Time	Block	Type	On screen
1	09:00–09:20	Mappings and service commitments	Demo	Anchor › STTM and service levels (two diagrams, nines table)
1	09:20–09:45	Activity 6 — Extend the contract	Challenge	Anchor › Activity 6 card
1	09:45–10:00	Break	Break	

© 2026 Decoded Limited · decoded.com

§	Time	Block	Type	On screen
1	10:00–10:15	Review and present a contract	Shareback + demo	VS Code Markdown editor, then Copilot/Claude, then the browser
2	10:15–10:45	Platform architecture: warehouse, lake, lakehouse and Databricks under the hood	Demo + shareback	Anchor › Platform architecture (five diagrams); the Delta and Spark demos
2	10:45–11:00	Break	Break	
3	11:00–11:25	Docker demonstration	Demo	Anchor › Docker; a terminal; the dev container’s Dockerfile
3	11:25–12:00	Challenge 7 — Docker practical	Challenge	Anchor › Challenge box; learners in challenges/v1/01-docker-intro
3	12:00–13:00	Lunch	Break	
4	13:00–13:15	Contract to code, how the tools fit together, Databricks setup	Demo	Anchor › Contract to code; Databricks (learners) and DuckDB (instructor)
4	13:15–14:05	Challenge 8 — Generate and investigate	Challenge	Anchor › Activity 8 card and prompts
4	14:05–14:15	Break	Break	
4	14:15–14:30	Debrief	Shareback	Anchor › Debrief (dbt flow diagram)

© 2026 Decoded Limited · decoded.com

§	Time	Block	Type	On screen
4	14:30–14:50	Review another pair’s generated code	Challenge	Anchor › Activity 8b card
4	14:50–15:00	Corrections and closing discussion	Shareback	Anchor › Before learners leave

© 2026 Decoded Limited · decoded.com

1. Mappings and service commitments

Welcome — yesterday’s contract is today’s input · 09:00–09:20

Purpose. Source-to-target mapping rule types; SLI, SLO and SLA as measure, ideal, tolerance; the indicators a product needs; availability in nines.

On screen. Anchor › STTM and service levels (two diagrams, nines table)

Instructor notes.

- Elicit rule types on the whiteboard; Catalyst returns a CG-specific list, say so.
- The nines table is in the anchor; say the month, week, day, hour, minute mnemonic.

Say.

- “An SLI is a number you can measure. An SLO is what you wish it were. An SLA is how far it may stray before you owe someone an explanation.”

Say.

- “Open the contract you made yesterday. Everything before lunch happens to that document.”
- “Hands up: who has written or reviewed source-to-target rules? Worked with an SLI? With an SLA? Usually most hands for SLAs and fewest for SLIs.”

Demo — STTM and service levels · 09:00–09:20

Ask at the whiteboard — STTM. Ask in order, write what the room says, add what is missing.

Ask	Expect, then add
What is a source-to-target mapping? What is its core?	An artefact showing where data comes from and goes to; the core is the transformation rules.

© 2026 Decoded Limited · decoded.com

Ask	Expect, then add
What kinds of rule can a column have?	Direct (copy), constant, lookup (a third table), aggregate (a mean), derive or transform (an operation; concatenate is a subtype). Ask Catalyst for CG's list.
What is in the design documentation set?	PRD, data contract, glossary or data dictionary, STTM, service levels.

Ask at the whiteboard — service levels. Ask in order, write what the room says, add what is missing.

Ask	Expect, then add
What do SLI, SLO and SLA stand for, and what do they mean?	Indicator: the measurable quantity. Objective: its ideal value. Agreement: the permissible departure and the consequences.
Give me indicators.	Delay from delivery time (which time zone?), latency, throughput, recovery time (service, data or level), completeness, accuracy, freshness, availability, support turnaround, a duty to explain.
What is availability, and what does 99.9% mean?	The share of time the service works, in nines. 90% is a month a year; 99% a working week; 99.9% a working day; 99.99% an hour; 99.999% five minutes.
How do providers get more nines?	Cloned infrastructure, global and local load balancing, a disaster-recovery site that already takes some traffic. Nines tell you about spare capacity.

Transition. Activity 6: put both into yesterday's contract.

If ahead or behind. Running late: skip how providers get nines; keep the table.

Activity 6 — extend the contract · 09:20–09:45

Purpose. Ask Catalyst for the transformation types and service measures, then add realistic STTM rules and commitments to a Day 1 contract.

On screen. Anchor › Activity 6 card

© 2026 Decoded Limited · decoded.com

Instructor notes.

- Transcribe the table's rule discussion and feed it in, as on Day 1.
- Collect one revised contract per table for the review block.

Say.

- "Be specific enough that someone else could implement the rule without asking you what it means."

Activity 6 — Extend the contract. Individual, then pairs or tables, 25 min; the card is in `activities/`.

Transition. Break; after it, one submitted contract on screen.

If ahead or behind. Overrun flows into the break; the review block uses whatever was submitted.

> ☕ Break · 15 min · back at 10:00

Shareback — review and present a contract · 10:00–10:15

Purpose. Review a submitted contract in the Markdown editor, then generate a self-contained HTML version with an overview, contents and diagrams.

On screen. VS Code Markdown editor, then Copilot/Claude, then the browser

Instructor notes.

- Open with 'Open with Markdown editor'; then the prompt in the anchor; use a fast model at low effort first and let the big model finish in the background.
- Open the HTML in the browser; show it is navigable and iterable.

Say.

- "Same file, two renderings. The AI does not care; you and your reviewers do."

Demo — Markdown editor, then a self-contained HTML page. Read Markdown properly, then turn the contract into one HTML page on a small model and on a big one.

1. Right-click a submitted contract, Open with..., Markdown editor (or Open Preview); edit one cell.
2. Say: this is how to read the long READMEs on Wednesday and Thursday.
3. In Claude Code (or Copilot) run the prompt below on a fast model at low effort.

© 2026 Decoded Limited · decoded.com

4. Open the result in the browser: contents, overview, diagrams.
5. Start the same prompt on the biggest model at high effort; let it run while you teach; compare when it finishes.
6. Say why the reasoning is hidden now (vendors stopped showing traces after competitors trained on them) and why the big model is slow.

Transition. Before we leave design: what makes an English rule unimplementable?

If ahead or behind. If generation stalls, open common/running-example/data-contract.html.

© 2026 Decoded Limited · decoded.com

2. Platform architecture

Demo — warehouse, lake and lakehouse; Databricks under the hood; Delta; Spark; the tools together · 10:15–10:45

Purpose. Warehouse vs lake vs lakehouse; where Databricks fits; Delta tables under the hood; what the Spark engine does; how the tools fit together.

On screen. Anchor › Platform architecture (five diagrams); the Delta and Spark demos

Instructor notes.

- Elicit each tool's role first, then the tools-together diagram.
- Run the Delta demo: files plus a log, a merge, time travel, OPTIMIZE; then the Spark demo with the UI open.
- Shareback: each table names one thing Databricks does that a plain file store cannot, and one thing dbt relies on Databricks for.

Say.

- "A lakehouse is a relational database split in two: files on S3, compute on Databricks, and a transaction log in between to get the guarantees back."

Ask at the whiteboard — warehouse, lake, lakehouse. Ask in order, write what the room says, add what is missing.

Ask	Expect, then add
What does a warehouse give you, and what does it cost?	Tables, SQL, transactions, strong consistency; proprietary storage you pay to scale; structured data only.
What is a data lake?	A cloud file system (S3) with any format in it: CSV, Parquet, JSON, images. Cheap, scales, no structure guaranteed, so every program must know every file.
What goes wrong with lakes?	No transactions: half-written files, duplicates on retry, no history, nobody knows what a file means. The data swamp.

© 2026 Decoded Limited · decoded.com

Ask	Expect, then add
What does a lakehouse add?	An open table format on the files (Delta: Parquet plus a log) so you get ACID, versions and schema; an engine that speaks SQL (Spark); a catalog for names, permissions and lineage.

Demo — Delta under the hood, on this laptop. A Delta table is Parquet files plus a log: the cell writes one and reads every version back.

1. Run the cell: it writes the vendor prices as a Delta table, appends a day, merges a correction, reads version 0, compacts, and reads the table from DuckDB.
2. Open the table folder the script names: the Parquet files and `_delta_log/*.json`.
3. Point at the merge output: the corrected row replaced, the old file kept, the log naming the files in each version.
4. Say: on Databricks the same table sits in S3 and Spark reads it; the format is open, which is why dbt, DuckDB, Spark and pandas all read it.

Demo — the Spark engine, locally. A local Spark shows the driver, executors, partitions and the shuffle that Databricks runs for you at scale.

1. Run the cell (about thirty seconds): a local Spark with four executors loads the vendor prices multiplied up and runs a SQL aggregate.
2. Find *Exchange* in the printed physical plan: that is the shuffle between stages.
3. Run it again with `--hold 90` and open http://localhost:4040: Jobs, Stages, the SQL tab.
4. Say: Databricks is this on many machines, with Delta and Unity Catalog around it, and you never configure it.

Ask at the whiteboard — the paved path, tool by tool. Ask in order, write what the room says, add what is missing.

Ask	Expect, then add
Which technologies are on the paved path?	Databricks, dbt, S3, Airflow with Astronomer, Cosmos, Docker, Git and GitHub, Harness, Unity Catalog, Marketplace, Catalyst, Datadog.
What does dbt provide? Databricks? S3?	The transformation rules and the tests. The compute that runs the SQL as Spark. Storage, as Delta files.
Airflow? Astronomer? Cosmos?	Timing and triggering. Managed Airflow, vended per team. The adapter that renders a dbt project as Airflow tasks.

© 2026 Decoded Limited · decoded.com

Ask	Expect, then add
Docker? Git? Harness? Datadog?	The environment for development and deployment. Version control whose pull-request events trigger release. The CI/CD pipeline that deploys on merge. Monitors and alerts on the runs.
When may you leave the path?	With a real reason (an API served from S3, a specialist engine); then you own every problem yourself.

Shareback — one minute per table · 10:15–10:45

Transition. Break, then Docker.

If ahead or behind. Behind: show the tools-together diagram and the Delta demo only; the timeline and Spark are optional.

> ☕ Break · 15 min · back at 11:00

© 2026 Decoded Limited · decoded.com

3. Docker

Demo — containers, the challenge's five steps, the dev container's Dockerfile · 11:00–11:25

Purpose. Dependencies, reproducibility, isolation and resource limits; Dockerfile to image to container; the dev container and its terminal.

On screen. Anchor › Docker; a terminal; the dev container's Dockerfile

Instructor notes.

- Finish the HTML review first if the big model has finished.
- Run the anchor's cells: run an image, look inside, build, run, tag.
- Show the training repo's Dockerfile: FROM, COPY, RUN, USER, ENV. Say EDO maintains it.

Say.

- "Your terminal in VS Code is inside a container. That is why dbt and astro are there and why your laptop's Python is not."

Demo — the five steps of Challenge 7, in cells. The challenge's own six-holding app (`demos/docker/portfolio-summary/`), run and built in cells before learners do it.

1. Check Docker Desktop is running: the whale in the menu bar, or `docker info`.
2. Cell 1: run `python:3.11-slim` and print its Python; the container exits when the command does.
3. Cell 2: write a file inside one container and list it from another: its own filesystem, gone with the container.
4. Cell 3: read the app's Dockerfile: FROM, WORKDIR, COPY, CMD.
5. Cell 4: build twice; the second build is all CACHED.
6. Cell 5: run the app, then run it with a mounted holdings file: same image, different data.
7. Cell 6: tag the image: two names, one ID; that is how CG ships Airflow and dbt images.
8. After each cell, say what learners will see at the same step of `challenges/v1/01-docker-intro`.

Demo — the dev container's Dockerfile. The learners' dev container is a Dockerfile they can now read.

© 2026 Decoded Limited · decoded.com

1. Open `reference/DataCreators-client/dev-container/edo-training-data-products/.devcontainer/Dockerfile`, the file VS Code builds for learners (the cell below prints its head).
2. Point at the two `FROM` lines: a flyway stage, then CG's Python 3.13 image from the internal registry.
3. Point at the `RUN curl` lines: the Astro CLI, starship and mise; then `mise install databricks-cli`.
4. Point at `COPY` of the setup scripts and `ENV VIRTUAL_ENV=/opt/venv` with its `bin` on `PATH`.
5. Say: EDO maintains this; you never will; but now you can read it.
6. Open Docker Desktop: Images, then Settings and Resources, briefly.

Transition. Challenge 7 in the dev container until lunch.

Challenge 7 — Docker practical · 11:25–12:00

Purpose. Run, inspect, build and rerun a small image; the one-command gate.

On screen. Anchor › Challenge box; learners in challenges/v1/01-docker-intro

Instructor notes.

- Sections 1–5 of the README, about 35 minutes; section 6 and the variations for fast finishers.
- Keep a working example ready for anyone blocked by engine access; pair them.

Say.

- "Why did the container stop, and what survives when you run a fresh one?"

Activity 7 — Docker practical. Individual or pairs, 35 min; the card is in `activities/`.

Challenge — Docker practical. `challenges/v1/01-docker-intro/README.md` · 35 min.

Transition. Lunch.

If ahead or behind. Experienced people: multi-stage builds, resource limits, Kubernetes questions.

> 🍽 Lunch · 60 min · back at 13:00

© 2026 Decoded Limited · decoded.com

4. Contract to code

Demo — where output lands, and one minute of live generation · 13:00–13:15

Purpose. Brief Challenge 8; show authentication, the training schema and where output lands.

On screen. Anchor › Contract to code; Databricks (learners) and DuckDB (instructor)

Instructor notes.

- Learners: mise run setup:databricks, dpc-check, the catalog training_dev and schema training_. Instructor: the same thing in DuckDB with information_schema.
- Run the first minute of a generation live, then open the prepared eod_prices project.

Say.

- “Most of the Databricks UI you will never need on the paved path. You write dbt in files; the UI is where the tables appear.”

Demo — where output lands: Databricks for learners, DuckDB here. Learners’ tables land in Databricks under their own schema; here the same tables land in a DuckDB file.

1. Learners: `mise run setup:databricks` (browser OAuth; Enter through the prompts), then `dpc-check`.
2. Databricks UI: Catalog, `training_dev`, `training_<username>`; every dbt table appears there.
3. Say: on the paved path you rarely open this UI; you write dbt in files and commit.
4. Here: run the cell below; it lists the same schema in the DuckDB file once the prepared project is built.

Demo — one minute of live generation. Start the generation live, stop after a minute, then open the finished project.

1. In Claude Code: `/setup-dbt-project` with `common/running-example/data-contract.md` and `sttm.md` attached.
2. Let it run for a minute and narrate what it reads first.
3. Stop it and open `demos/contract-to-code/eod_prices/models/`: every CTE comments the STTM rule it implements.
4. Open `tests/` and `schema.yml` there: the tests mirror the contract’s quality rules.
5. Open `eod_prices_daily.py` in `../day4-airflow/demos/airflow_local/dags/eod_prices/dags/`: the schedule comes from the SLA.

© 2026 Decoded Limited · decoded.com

Transition. Challenge 8.

Activity 8 — generate and investigate · 13:15–14:05

Purpose. Generate dbt and Airflow code from the contract, explain it, run as far as possible, and bring one discovery and one correction.

On screen. Anchor › Activity 8 card and prompts

Instructor notes.

- Pairs who finish choose one extension question from the card.
- Experienced learners: the bitemporal macro or Airflow-to-Datadog logging; keep findings short.

Say.

- “Run what you can. What you cannot run, read until you can explain it.”

Activity 8 — Generate and investigate. Pairs, 50 min; the card is in `activities/`.

Transition. Break, then the debrief in the stated order.

If ahead or behind. If Databricks auth is widespread trouble, let pairs generate and read without running; the debrief is unchanged.

> ☕ Break · 10 min · back at 14:15

Debrief — in this order · 14:15–14:30

Purpose. dbt as ELT; SELECT, Jinja and YAML; views and tables; OLTP and OLAP; the commands; then DAGs, operators, blueprints and task groups.

On screen. Anchor › Debrief (dbt flow diagram)

Instructor notes.

- Follow the anchor’s order; ask before telling.
- Land the 80/20 message: the contract produced most of the code.

Say.

© 2026 Decoded Limited · decoded.com

- “If eighty percent of the code came from the contract, then the interview and the contract were the job.”

Ask at the whiteboard — debrief, in this order. Ask in order, write what the room says, add what is missing.

Ask	Expect, then add
What is dbt?	An ELT tool: load first (seed, land), transform in the warehouse. Models are SELECT statements with Jinja {{ }} replaced by SQL; dbt turns each into CREATE TABLE or VIEW plus inserts. The model name is the file name.
What is in the YAML?	Sources, descriptions, tests, materialisation. Views for staging and intermediate; tables for marts, unless a downstream consumer needs the intermediate as a table.
OLTP vs OLAP?	Transactions (retail ledgers, debit and credit as one unit, linearisable) vs analysis (bulk insert, bulk read). dbt lives in OLAP.
Which commands?	debug (connection), seed (CSV into the warehouse; a learning tool), run, test, build (run and test in graph order), docs.
And Airflow?	You define a DAG: a collection of tasks with order, branches and fan-out, in Python. Operators connect to systems: the Databricks operator runs a job; the Cosmos dbt task group renders each dbt stage as a task. At CG, blueprints fill Python templates from YAML.
How well did contracts translate?	Rooms say ‘very straightforwardly’; one pair’s product became internal and external models with a union view, unprompted. That is the 80/20: the contract did the work; the 20 is review, tests, edge cases.

Transition. Review another pair’s code.

Activity 8b — review another pair’s generated code · 14:30–14:50

Purpose. Trace one contract rule into code, find one assumption, propose one check and say what happens when it fails.

© 2026 Decoded Limited · decoded.com

On screen. Anchor › Activity 8b card

Instructor notes.

- Five minutes explaining, ten reviewing, five discussing.
- Code that cannot run is still reviewable; everyone explains their own rule.

Say.

- “Find the line where the rule lives. Then find the assumption the rule does not mention.”

Activity 8b — Review another pair’s generated code. Pairs, swapped, 20 min; the card is in `activities/`.

Transition. One correction each, then close.

Close — corrections and closing discussion · 14:50–15:00

Purpose. Make one correction and share its reason; record questions for the dbt practical.

On screen. Anchor › Before learners leave

Instructor notes.

- Separate access problems from problems in the generated code when you write them up.
- Tomorrow’s launch script is in the Day 3 anchor; tell learners to open challenges/v1/02-dbt-pipeline tonight if they want a head start.

Say.

- “Tomorrow is self-paced: the README teaches dbt as you go, and I walk the room.”

Say.

- “Make one correction now, and say why. Then write down the questions you want answered tomorrow, and keep the access problems on a separate list from the code problems.”

© 2026 Decoded Limited · decoded.com

Before learners leave

- [ ] The revised mapping and commitments, the generated dbt and Airflow project, the commands that worked and the unresolved setup issues, saved per pair.
- [ ] Each learner can locate one output table and explain one transformation in it.
- [ ] Access problems recorded separately from problems in the generated code, with an owner for each.
- [ ] Anyone still blocked on the dev container or Databricks has a platform-support contact for tonight.

© 2026 Decoded Limited · decoded.com